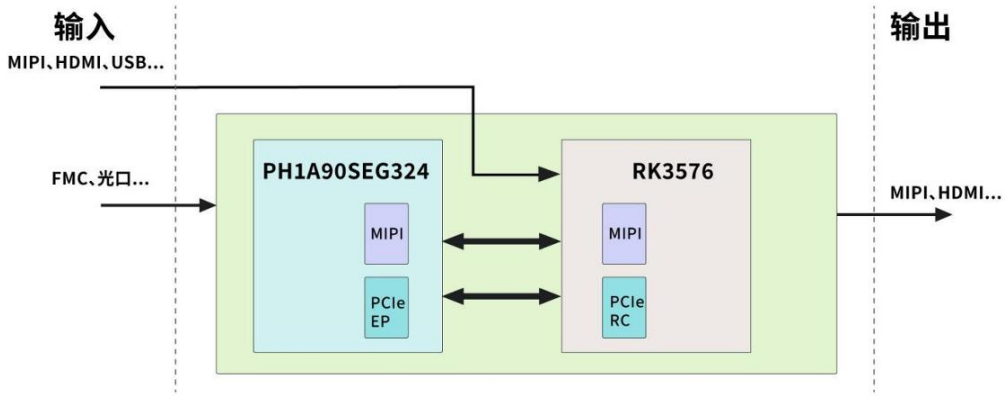
第一章 MIPI-ISP-MIPI 链路测试

1 简介

本项目基于自研异构核心板，融合安路 PH1A90SEG324FPGA 与瑞芯微 RK3576 处理器，构建 MIPI 采集 +FPGA 预 处理 +MIPI 输出 双链路高速互联的完整异构视频处理架构。 系统支持通过 FMC 扩展多路 MIPI-CSI 接口，兼容主流工业相机，由 FPGA 实现高吞吐量图像采集与实时 ISP 预 处理；FPGA 与 ARM 平台间基于 MIPI 双链路的自定义高速传输协议，实现稳定、低延迟的数据交互；

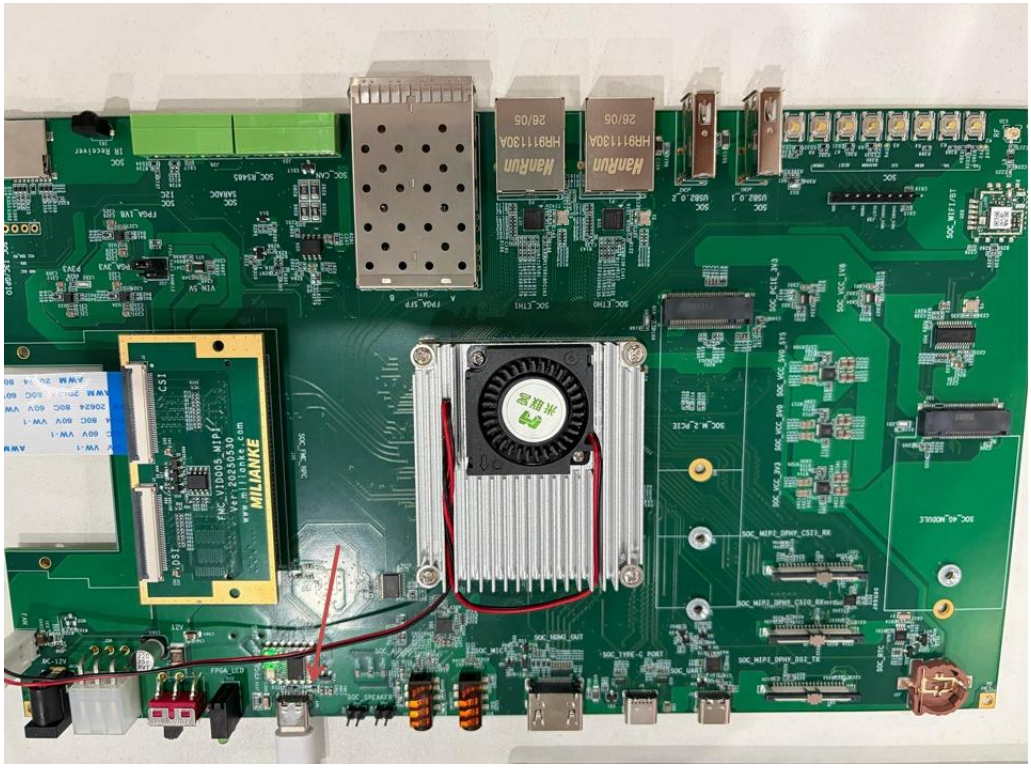
2 框图



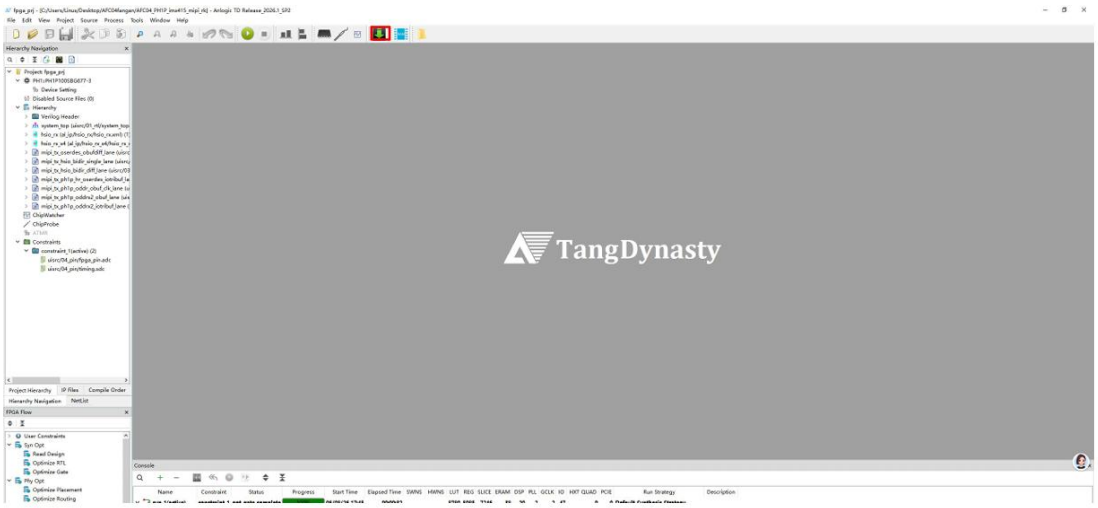
3 烧写 FPGA 端程序

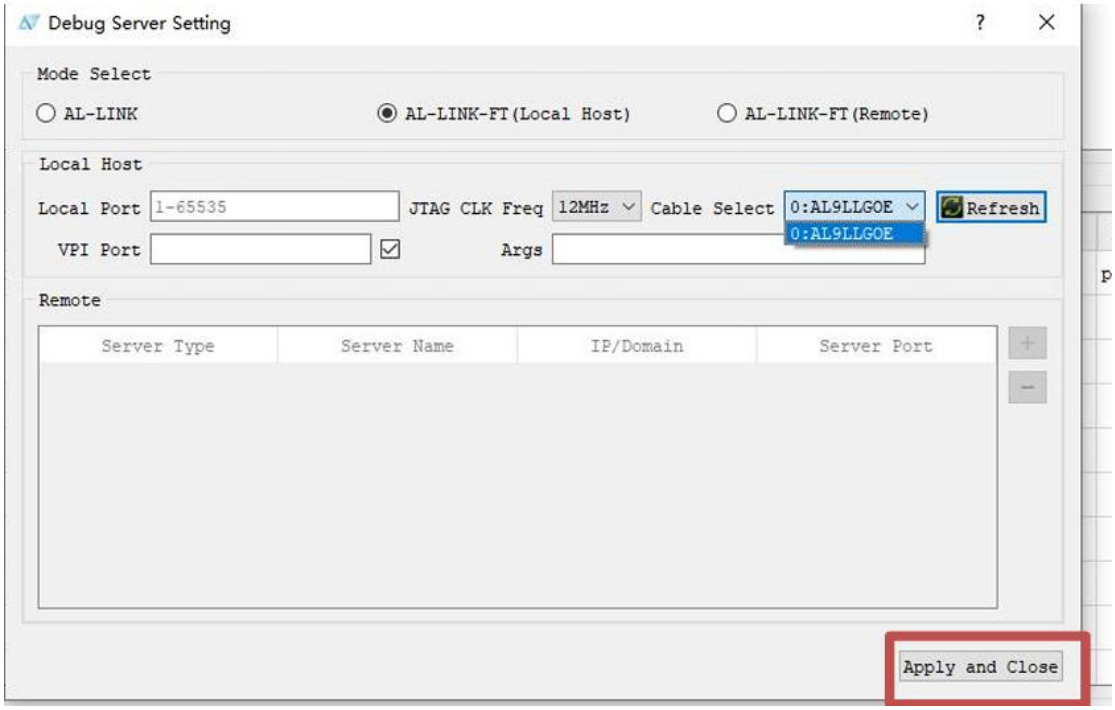
先看提供文件夹给的 readme.txt 完成镜像的烧录

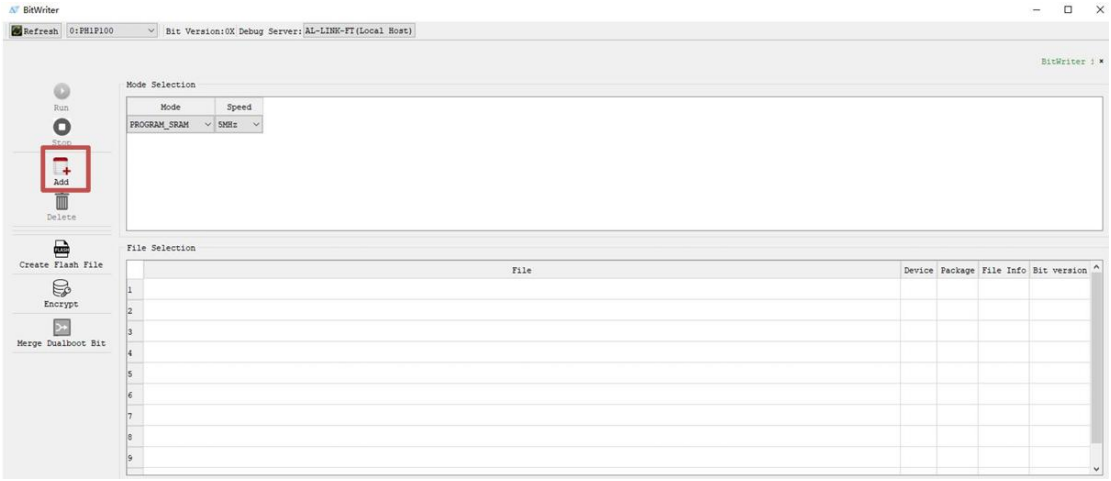
准备好本文档的附件，解压到纯英文路径后后找到 fpga_prj.al，再使用 TD 打开开发板确保断电的情况下，接入 jtag 再上电开机：



左侧栏连接硬件：

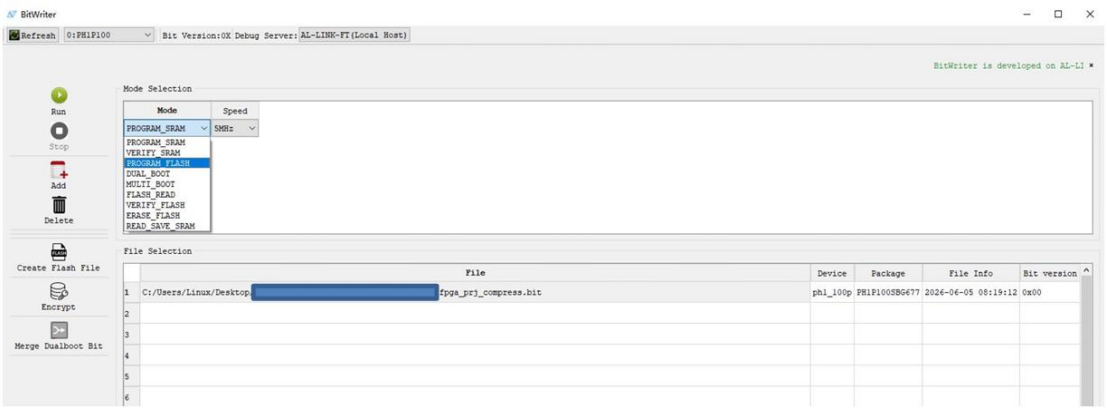


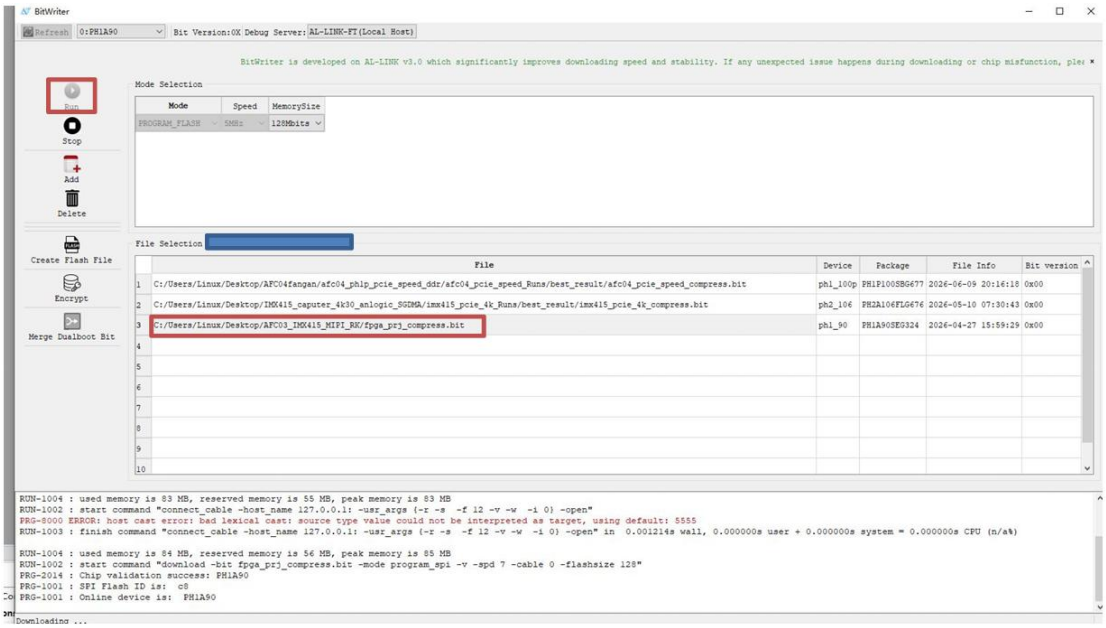




选择附件下名为 fpga_prj_compress.bit 的文件

选择 mode



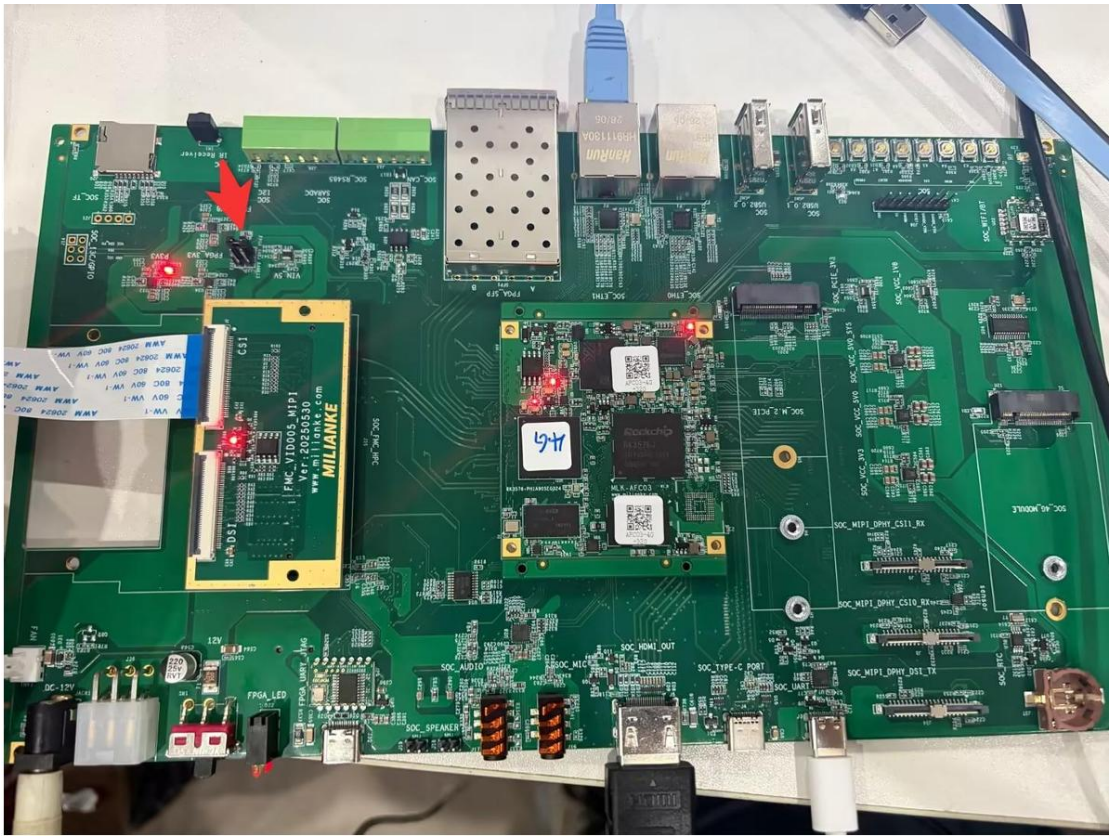


点击 run 然后等烧录完下方会出现 succeeded

在官网下载 https://www.uisrc.com/t-6245.html 3576debian 镜像

然后把提供的 boot.img 烧录至 boot 分区,开发板直接上电，烧录分区镜像不要按 boot 键

加跳线帽



4 测试实验

输入指令：gst-launch-1.0 v4l2src device=/dev/video22 ! kmssink

```
root@linaro-alip:/# gst-launch-1.0 v4l2src device=/dev/video22 ! kmssink
```



第二章 YOLO 识别添加

1 文件拷贝到开发板

把附件中提供的 rock 文件夹拷贝放到开发板下，这里我以放到/home 下为例子

验证：

```
cd /home/rock/yolo
ls
```

```
root@linaro-alip:/home/rock/yolo# ls
det.jpg  rk3576_yolov5s_relu.py  test_frame.jpg  yolov5s_relu.rknn
root@linaro-alip:/home/rock/yolo# cd /home/rock/whls
```

```
cd /home/rock/whls
ls
```

```
root@linaro-alip:/home/rock/whls# ls
numpy-2.2.6-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
opencv_python-5.0.0.93-cp37-abi3-manylinux2014_aarch64.manylinux_2_17_aarch64.whl
opencv_python_headless-5.0.0.93-cp37-abi3-manylinux2014_aarch64.manylinux_2_17_aarch64.whl
psutil-7.2.2-cp36-abi3-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl
rknn_toolkit_lite2-2.3.0-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
ruamel_yaml-0.19.1-py3-none-any.whl
ruamel_yaml_clib-0.2.15-cp311-cp311-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl
```

2 开发板环境配置（离线安装）

2.1 安装 Python 依赖

```
# 确认 Python 版本是 3.11
python3 --version
# 必须输出 Python 3.11.x

# 确认目标目录存在
python3 -c "import site; print(site.getsitepackages())"
# 通常包含 /usr/local/lib/python3.11/dist-packages

# 进入 whl 所在目录
cd /home/rock/whls
# 按顺序逐个解压安装（复制以下全部内容一次性粘贴）
unzip -o numpy-2.2.6-cp311-*.whl                -d /usr/local/lib/python3.11/dist-packages/
unzip -o ruamel_yaml_clib-0.2.15-cp311-*.whl    -d /usr/local/lib/python3.11/dist-packages/
unzip -o ruamel_yaml-0.19.1-*.whl               -d /usr/local/lib/python3.11/dist-packages/
```

```
unzip -o psutil-7.2.2-*.whl                    -d /usr/local/lib/python3.11/dist-packages/
unzip -o rknn_toolkit_lite2-2.3.0-cp311-*.whl  -d /usr/local/lib/python3.11/dist-packages/
```

2.2 安装 OpenCV

开发板原先预装的是 headless 版（无显示功能），必须替换为完整版：

```
# 删除旧的 headless 版
rm -rf /usr/local/lib/python3.11/dist-packages/cv2
rm -rf /usr/local/lib/python3.11/dist-packages/opencv_python_headless*

# 安装完整版
unzip -o opencv_python-5.0.0.93-cp37-abi3-manylinux2014_aarch64.manylinux_2_17_aarch64.whl \
  -d /usr/local/lib/python3.11/dist-packages/
```

2.3 验证所有依赖

```
python3 -c "import numpy; print('numpy:', numpy.__version__); import cv2; print('opencv:', cv2.__version__); print('imshow:', hasattr(cv2, 'imshow')); import psutil; print('psutil:', psutil.__version__); from rknnlite.api import RKNNLite; print('RKNNLite: OK'); print('ALL OK')"
```

如果某一项报错，检查对应 whl 是否解压正确。

3 运行 YOLOv5s-Relu 实时推理

确认摄像头设备

```
# 确保 FPGA bit 已烧录
# 确保摄像头排线已连接

ls /dev/video*
# 应该能看到 /dev/video22（或类似编号）
```

运行指令

```
cd /home/rock/yolo
export DISPLAY=:0
python3 rk3576_yolov5s_relu.py
```


HL2023-HDMI

第三章 添加自启动

1 文件

```
/home/rock/yolo/rk3576_yolov5s_relu.py #识别主程序
/home/rock/yolo/start_yolov5.sh #启动脚本
/etc/rc.local   #开机自启
```

2 配套保护

2.1 panic 自动重启（防永久卡死）

内核崩溃（panic）后 5 秒自动重启，不会一直卡住：

```
echo "kernel.panic = 5" > /etc/sysctl.d/99-panic.conf
sysctl -p /etc/sysctl.d/99-panic.conf
sysctl kernel.panic
```

最后一行输出 kernel.panic = 5 即成功。

2.2 启用 rc-local 服务（开机执行 rc.local）

```
chmod +x /etc/rc.local
systemctl enable rc-local
systemctl status rc-local
```

2.3 no_yolo 开关（一键禁用自启）

```
touch /root/.no_yolo     # 禁用自启（下次开机不跑 YOLO）
rm -f   /root/.no_yolo    # 恢复自启
```

3 重启验证

reboot 重启 45 秒后自动弹全屏窗口

```
rm -f /root/.no_yolo    # 先确认开关已删
reboot
pkill -9 -f rk3576_yolov5s_relu.py   #结束自动识别
```